

Experiment No. 2

Experiment Name

Contrast Enhancement and Histogram-Based Image Processing using Python and OpenCV

part-1 Python code

```
In [1]: #step-1
import cv2
import numpy as np
import matplotlib.pyplot as plt
#step-2
img = cv2.imread("image.jpg.png")
# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis("off")
plt.show()
#step-3
plt.hist(gray.ravel(), 256, [0, 256])
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()
#step-4
min_val = np.min(gray)
max_val = np.max(gray)
stretched = ((gray - min_val) / (max_val - min_val) * 255).astype(np.uint8)
plt.imshow(stretched, cmap='gray')
plt.title("Contrast Stretched Image")
plt.axis("off")
plt.show()
#step-5
equalized = cv2.equalizeHist(gray)
plt.imshow(equalized, cmap='gray')
plt.title("Histogram Equalized Image")
plt.axis("off")
plt.show()
#step-6
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

clahe_img = clahe.apply(gray)

plt.imshow(clahe_img, cmap='gray')
plt.title("CLAHE Enhanced Image")
plt.axis("off")
plt.show()
# step-7
```

```

plt.figure(figsize=(10, 6))
plt.hist(gray.ravel(), 256, [0, 256], alpha=0.5, label="Original")
plt.hist(equalized.ravel(), 256, [0, 256], alpha=0.5, label="Equalized")
plt.hist(clahe_img.ravel(), 256, [0, 256], alpha=0.5, label="CLAHE")
plt.title("Histogram Comparison")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.legend()
plt.show()

#step-8
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(stretched, cmap='gray')
plt.title("Contrast Stretching")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(equalized, cmap='gray')
plt.title("Histogram Equalization")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(clahe_img, cmap='gray')
plt.title("CLAHE")
plt.axis("off")
plt.tight_layout()
plt.show()

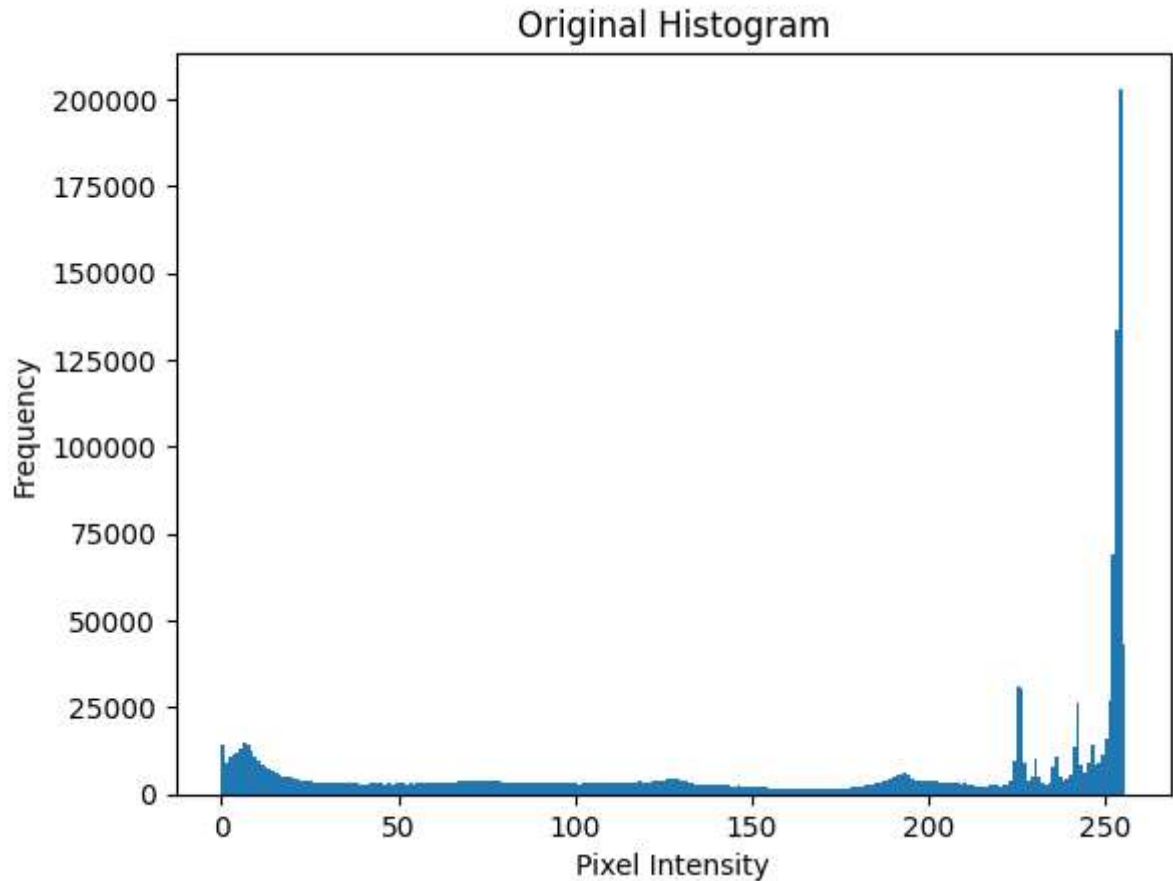
#step-9
#pip install opencv-python numpy matplotlib

```

Original Image



C:\Users\vivek\AppData\Local\Temp\ipykernel_14708\606085097.py:14: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.
 plt.hist(gray.ravel(), 256, [0, 256])



Contrast Stretched Image



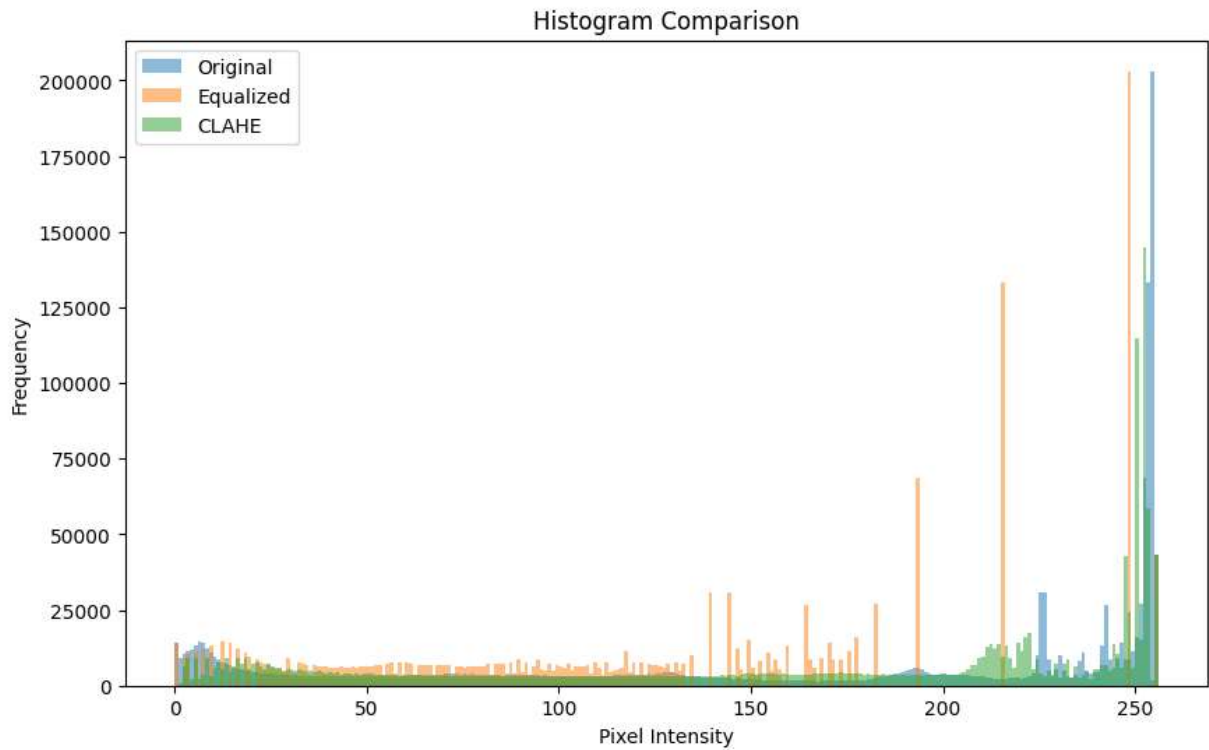
Histogram Equalized Image

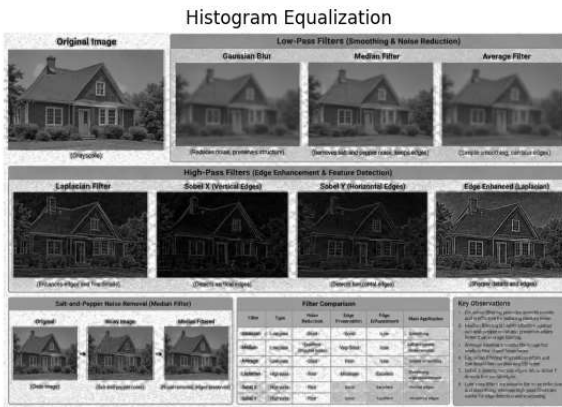


CLAHE Enhanced Image



```
C:\Users\vivek\AppData\Local\Temp\ipykernel_14708\606085097.py:47: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.  
plt.hist(gray.ravel(), 256, [0, 256], alpha=0.5, label="Original")  
C:\Users\vivek\AppData\Local\Temp\ipykernel_14708\606085097.py:48: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.  
plt.hist(equalized.ravel(), 256, [0, 256], alpha=0.5, label="Equalized")  
C:\Users\vivek\AppData\Local\Temp\ipykernel_14708\606085097.py:49: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.  
plt.hist(clahe_img.ravel(), 256, [0, 256], alpha=0.5, label="CLAHE")
```





Part - 2 Question

1. What is a digital image? Differentiate between grayscale and color images.

A digital image is a two-dimensional representation of a visual scene made up of small elements called **pixels**. Each pixel contains intensity or color information.

A **grayscale image** contains only intensity information and generally has one channel, while a **color image** contains multiple channels such as Red, Green, and Blue.

Grayscale Image

Contains one channel.

Represents brightness/intensity.

Values generally range from 0–255.

Requires less storage.

Easier and faster to process.

Color Image

Usually contains three channels.

Represents color information.

Each color channel generally ranges from 0–255.

Requires more storage.

More computationally expensive.

2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

RGB, BGR, HSV, and LAB are different ways of representing color information in an image.

- **RGB** → Red, Green, Blue. Commonly used for image display.
- **BGR** → Blue, Green, Red. OpenCV normally reads images in this format.

- **HSV** → Hue, Saturation, Value. Useful for color detection and segmentation.
- **LAB** → Lightness, A, B. Useful for color analysis.

Color Space	Components	Main Use
RGB	Red, Green, Blue	Image display
BGR	Blue, Green, Red	OpenCV image processing
HSV	Hue, Saturation, Value	Color detection/segmentation
LAB	Lightness, A, B	Color analysis

3. What is the purpose of converting an image to grayscale before further processing?

Converting an image to grayscale changes a color image from three channels into a single intensity channel.

It is useful because:

- It reduces computational complexity.
- It requires less memory.
- It simplifies image processing.
- It removes unnecessary color information.
- It improves processing speed.
- It is commonly used before edge detection, thresholding, and segmentation.

In OpenCV:

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

4. Explain the concept of image complement (negative image) and mention its practical applications.

An image complement, also called a **negative image**, is produced by reversing the intensity values of the original image.

For an 8-bit image:

Negative = 255 – Original Pixel

For example:

```
Original pixel = 50
Negative = 255 - 50
          = 205
```

Therefore, dark regions become bright and bright regions become dark.

Applications include:

- Medical image visualization.
 - X-ray image processing.
 - Image enhancement.
 - Photograph processing.
 - Highlighting details in dark regions.
-

5. Differentiate between image resizing, cropping, and scaling.

Image resizing changes the dimensions of an image to a specified width and height.

Cropping selects a particular portion of an image and removes the remaining portions.

Scaling changes the size of an image using a scaling factor.

Operation	Meaning
Resizing	Changes image dimensions to specified values.
Cropping	Selects a particular region of an image.
Scaling	Changes image size according to a scale factor.

Example of resizing:

```
resized = cv2.resize(image, (400, 300))
```

6. What is a Region of Interest (ROI), and why is it important in computer vision?

A **Region of Interest (ROI)** is a specific portion of an image selected for processing or analysis.

For example:

```
roi = image[y1:y2, x1:x2]
```

ROI is important because:

- It focuses processing on the required area.
- It reduces computational cost.
- It improves processing efficiency.
- It helps in object detection.
- It is useful in face detection and image segmentation.

For example, in face recognition, only the region containing the face can be processed instead of the entire image.

7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

OpenCV (Open Source Computer Vision Library) is widely used for image processing and computer vision because it provides many optimized functions and algorithms.

Advantages of OpenCV include:

- It is open-source and free.
- It supports Python.
- It provides image and video processing.
- It supports image reading and writing.
- It provides resizing, rotation, cropping, and flipping functions.
- It supports different color spaces.
- It provides filtering and enhancement techniques.
- It supports feature detection and extraction.
- It can be used for real-time computer vision applications.

Example:

```
import cv2

image = cv2.imread("image.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

8. Explain how image resolution and pixel intensity influence image quality.

Image resolution refers to the number of pixels present in an image. For example, 1920 × 1080 represents the width and height of an image in pixels.

Higher resolution generally provides more visual details, while lower resolution can result in loss of detail or pixelation.

Pixel intensity represents the brightness of a pixel.

For an 8-bit grayscale image:

0 → Black
255 → White

Pixel intensity affects the brightness and contrast of an image.

Therefore:

- Higher resolution → More spatial details.
 - Lower resolution → Less detail.
 - Higher intensity → Brighter pixels.
 - Lower intensity → Darker pixels.
 - Intensity distribution affects image contrast.
-

9. Mention five real-world applications where basic image preprocessing is an essential step.

Basic image preprocessing is used in many real-world computer vision applications.

- **Medical Imaging** – X-rays, CT scans, and MRI images.
- **Face Recognition** – Face images are resized, converted, and enhanced before recognition.
- **Autonomous Vehicles** – Camera images are processed for detecting vehicles, pedestrians, roads, and signs.
- **Industrial Inspection** – Used for detecting cracks, scratches, and manufacturing defects.
- **Object Detection** – Images are resized, normalized, and processed before object detection.

Other applications include:

- OCR and document recognition.
- Satellite image analysis.
- Robotics.
- Security systems.
- Image classification.

10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Image preprocessing improves the quality and consistency of image data before it is provided to feature extraction algorithms or deep learning models.

Common preprocessing techniques include:

- **Resizing** – Converts images to a common size.
- **Grayscale conversion** – Reduces unnecessary color information.
- **Normalization** – Scales pixel values to a standard range.
- **Noise reduction** – Removes unwanted variations.
- **Contrast enhancement** – Makes important features more visible.
- **Color-space conversion** – Uses an appropriate representation such as HSV or LAB.

For example:

```
image = cv2.resize(image, (224, 224))  
normalized = image / 255.0
```

These techniques can:

- Improve image quality.
- Reduce unnecessary information.

- Standardize input data.
- Reduce computational complexity.
- Improve feature extraction.
- Make model training more stable.
- Help models learn useful patterns more effectively.

Therefore, **image preprocessing is an important step before feature extraction, machine learning, and deep learning.**